VCC视频会议 项目档案（分册版）

> **分册序号：** 第 8 册 / 共 12 册
> **分册标题：** 第八部分 部署落地与公开发布
> **分册说明：** 本册为 VCC视频会议 项目档案的分册版本，内容与主档案（VCC视频会议项目档案.md）保持一致，仅做结构分册处理，便于按需查阅。
> **主档案位置：** `VCC视频会议项目档案.md`

📑 本册目录

- 第八部分 部署落地与公开发布
  - 8.1 部署形态与路线选型
  - 8.2 方案 A：容器化部署
  - 8.3 方案 B：裸机 + mTLS
  - 8.4 质量门禁与冒烟体系
  - 8.5 数据库迁移、备份与回滚
  - 8.6 GitHub 公开发布
  - 8.7 作品链接与对外呈现
  - 8.8 部署与发布侧的缺陷台账

第八部分 部署落地与公开发布

> 本部分为 2026-10-22 增补，回答三个问题：**能不能真部署起来**、**能不能公开到 GitHub**、**有没有可展示的作品链接**。 所有结论均来自 `deploy/` 下脚本与实测记录、发布仓库 `D:\vcc-video-conference` 的 git 实际状态、以及 2026-10-21 全量审查报告，可逐条核验。

8.1 部署形态与路线选型

本项目的部署形态由架构决定：客户端是 Windows 32 位程序（Qt 5.12.11 / MinGW），服务端是 Linux 二进制且**只监听明文端口**，TLS 由外部网关终结——因此两条部署路线可以共用同一份服务端二进制，加 TLS 不需要改一行项目源码。

方案	适用场景	优点	代价	落地状态
A. Docker Compose	绝大多数场景	一条命令起全套（含 MySQL）、环境可复现、健康检查与依赖顺序固化成文件、换 tag 即回滚	需要 Docker	**已真机落地**（镜像 `wechat-server:2026.10.16-3`）
B. 裸机 + systemd + nginx mTLS	不便装容器的虚拟机	依赖最少、性能无损耗、systemd 托管自启与限流重启	环境手工维护、换机要重装	脚本齐备且在本地 Docker 实验室用同一份 nginx 配置验证过，但目标虚拟机拿不到 IP，**未真机落地**
C. 云服务器 + Compose	需公网或异地访问	有公网 IP、多端可连	必须补 TLS 与鉴权、需配安全组	未做
D. 只分发客户端、服务端跑本机	演示	零成本	关机即停，不可当正式部署	未做

选型判据不是偏好问题：要"环境可复现 + 一键回滚"选 A；不便装容器但要求公网 mTLS 选 B；内网用 A、对外边界用 B 的网关形态可以组合。

端口与暴露面策略

端口	用途	绑定	说明
8000	明文业务（客户端三路连接）	`127.0.0.1`	`${SERVER_BIND:-127.0.0.1}`，**不要改成 0.0.0.0**，否则无鉴权的二进制协议暴露到局域网
8443	mTLS 网关	对外	`ssl_verify_client on` + `ssl_crl`，无证书直接拒绝
8080	CRL 发布	仅内网	CRL 本身不含敏感信息，但没必要暴露公网

目标环境前置条件：Ubuntu 22.04 / x86_64 / ≥2 核 4GB / 磁盘 ≥20GB；Docker 20.10+ 与 Compose v2（实测环境 Docker Desktop 29.8.2 + Compose v5.6.0，WSL2 后端）；冒烟脚本需要 Python 3.6+。

> **环境级坑（实测）**：Docker 引擎起不来、报 `npipe:////./pipe/dockerDesktopLinuxEngine` 找不到时，**换 Docker 版本没用**。根因是引擎必须调用的 `C:\Windows\system32\wsl.exe` 被安全策略/第三方安全软件拦截，或 WSL 平台未安装。处置顺序：放行 `wsl.exe` → 确认 `wsl`

> `-l -v` 有分发 → 再重启 Docker。镜像拉取超时的解法是给 `~/.docker/daemon.json` 配 `registry-mirrors`。

8.2 方案 A：容器化部署

8.2.1 一键部署的执行顺序

```
cd deploy
cp .env.example .env          # 填写 DB_PWD / MYSQL_ROOT_PWD / CLIENT_SERVER_IP
./deploy.sh                   # 前置检查 → prepare → 构建 → up → 等 healthy → 隔离冒烟
./deploy.sh --tls             # 额外拉起 8443 mTLS 网关（公网必选，缺证书自动 gen-certs）
./deploy.sh --no-build        # 仅重启，不重新构建
```

`deploy.sh` 前置检查缺一项即退出：docker 存在、compose v2 存在、`.env` 存在、`DB_PWD` / `MYSQL_ROOT_PWD` 非空且不是占位值「请改成强口令」。

8.2.2 环境变量清单

变量	必需	默认	说明
`DB_NAME`	✅	`wechat`	库名。**部署时必须显式注入**——源码 `_DEF_DB_NAME` 的默认值与实际库不一致
`DB_USER`	✅	`wechat`	应用账号，不要用 root
`DB_PWD`	✅🔒	—	运行期注入，不进镜像层
`DB_IP`	✅	`mysql`	裸机改为 `127.0.0.1`
`MYSQL_ROOT_PWD`	✅🔒	—	仅容器初始化与运维
`SERVER_PORT`	⬜	`8000`	宿主映射端口（容器内仍 8000）
`SERVER_BIND`	⬜	`127.0.0.1`	明文端口绑定地址
`IMAGE_TAG`	✅	`latest`	回滚锚点，按 `YYYY.MM.DD-N` 命名
`CLIENT_SERVER_IP`	✅	—	写进客户端 `config.ini` 的服务端地址

8.2.3 镜像的两个关键设计

- **口令运行期注入**：`packdef.h` 用 `_db_env()` 从环境变量读 `DB_*`，compose 用 `environment:` 注入，镜像层里没有口令。早期"构建期用 `sed` 把口令写进源码再编译"的做法已废弃——它把口令固化进了镜像层。
- **双重启动等待**：容器 `entrypoint` 先 `nc -z mysql 3306` 循环等待，再由 `depends_on: service_healthy` 兜底。单靠后者不够：MySQL 端口开了但账号未初始化时，服务端 `Open()`

失败会直接 `exit(-1)`。

8.2.4 健康检查必须是业务级

探针方式	能发现的问题
`nc -z` 探端口	只能发现"进程没了"
`health-probe` 发 LOGIN_RQ 等 LOGIN_RS	还能发现**线程池/epoll 阻塞**与 **MySQL 不可用**——这两种情况下服务端不会回包，探针超时转 unhealthy

这也是 `health-probe.cpp` 存在的理由。日志为空不是故障：服务端所有 `printf` 都被注释掉，`err_str()` 在致命路径上是零日志静默 exit。

8.3 方案 B：裸机 + mTLS

8.3.1 拓扑

```
Qt 客户端 ──127.0.0.1:8000──> stunnel(本机) ══TLS1.2/1.3 + 客户端证书══> nginx:8443 ──>
                                                                          │
                                                          CRL:8080 ──────┘（发布吊销列表）
```

8.3.2 证书体系

项	改造前（方案 A 自签）	改造后（方案 B）
服务端证书	自签 CA 签发	**公共可信 CA**（Let's Encrypt DNS-01，不需公网 80 端口，支持泛域名）
客户端证书	所有人共用一张	**每人一套私钥 + 证书**，姓名/工号/邮箱写入 subject，p12 分发并登记台账
原自签 CA	又签服务端又签客户端	**降级为客户端专用根 CA**（`ca-init.sh --reuse-root` 可复用，避免重签全部客户端）
吊销	无	**CRL**，网关侧强校验，吊销后下一次握手即失效

8.3.3 反直觉陷阱：握手成功不能作为 mTLS 生效的判据

TLS 1.3 的 post-handshake authentication、以及 nginx 在 TLS 1.2 下的实现，都是**先完成握手、再校验客户端证书**。因此无证书的客户端 `wrap_socket` 也会返回成功，随后才被 alert 断开。

判据	结论
❌ 看"握手是否成功"	第一版反向用例因此误报"鉴权失效"
✅ 看字节有没有真的穿过去	反向用例在 TLS 1.2/1.3 各发一个字节做转发探测；正向用例额外发一次不存在号码的登录请求（只 SELECT 零写库）确认链路真通

网关日志才是真相：`client_cert=SUCCESS up=172.18.0.2:8000`（有证书，真转发）对 `client_cert=NONE up=-`（无证书，零字节转发）。

> **方法论沉淀**：这类"看起来通过了其实没验到"的检查，比没有检查更危险。安全项验收必须找到能证伪的判据。

8.3.4 生产加固 6 条

根 CA 私钥离线保存（网关机只留 `chain.crt`）；对 CRL 定时器加监控（**CRL 过期 = 全量断连**）；`inventory.csv` + `index.txt` 定期备份（丢了无法精确定位吊销对象）；人多后演进到 OCSP stapling；**业务层二次鉴权保留**（mTLS 只证明证书有效，账号密码/bcrypt 是叠加关系）；回滚只删 nginx 配置与 timer，**证书资产一律保留**，否则一次回滚会让证书体系不可恢复。

8.4 质量门禁与冒烟体系

8.4.1 门禁四步（fail fast，越便宜越靠前）

步	内容	为什么
1/4	`prepare.sh` + `prepare.sh --check`	构建上下文 `.staging` 若停在旧版本，编出来的镜像会"一切正常却不含最新修复"——最难发现的假绿
2/4	`docker compose build server`	编译错误最先暴露
3/4	`smoke.sh`（隔离栈）	功能冒烟，生产库零接触
4/4	`security_smoke.sh`（隔离栈）	注入回归：A 组打恶意包，B 组功能回归，跑完复核库行数

`ci-local.sh` 是唯一入口，支持 `--with-tls`（正/反两案握手）与 `--fast`（跳过构建）。`.github/workflows/ci.yml` **不写自己的逻辑，直接调用它**——否则必然出现"本地绿、CI 红"，且两套标准都会腐烂。

8.4.2 隔离冒烟：为什么不能"跑完删数据"

冒烟栈用 `docker compose -p wechat-smoke -f docker-compose.yml -f docker-compose.smoke.yml up` 起一次性环境，跑完 `down -v` 销毁。

被否决的替代方案是"跑完直连 MySQL 删掉冒烟数据"。**杀手锏理由**：verifier 生成的号码是 `139%08d`，最自然的清理条件 `DELETE FROM t_user WHERE tel LIKE '139%'` 会**误删真实用户**——139 是中国移动号段。此外隔离栈还顺带把"首次部署（空库 + schema.sql）"这条路径也测了，而直连清理永远测不到。

判断隔离成功的铁证是自增 ID：冒烟注册返回 `userid=1`（每次全新空库），而生产库自增已到 15；若误连生产库，返回值会是 16。这个数字比任何日志都可靠。

隔离栈三个必踩的坑：① `container_name` 全局唯一，必须显式改名；② `ports: []` **移不掉**主文件已声明的映射，要写同宿端口才能替换；③ 卷隔离靠 `-p` 前缀，无需覆盖 `volumes`。

8.4.3 测试覆盖的短板

服务端定义了 12 个协议处理器，自动化只覆盖 Register / Login 两个（**2/12 = 16.7%**）；音视频帧转发作为核心业务零自动化，回归只能靠人肉双端。这是当前最大的测试缺口。

8.5 数据库迁移、备份与回滚

机制	做法	为什么这样定
`schema.sql` 幂等	查 `information_schema.statistics` 判重 + 动态 SQL（存储过程用完即删）	MySQL 8.0 不支持 `CREATE INDEX IF NOT EXISTS`，裸机第二次部署必在 ERROR 1061 中断
两条路径同构	`schema.sql`（新装）与 `migrations/0001_init_t_user.sql`（升级）**md5 必须一致**（`653ae28e…`），`migrate.sh` 执行前比对	一旦分叉就会出现"新装的库和升级上来的库结构不同"，极难排查
版本表	`schema_migrations(version, name, md5, applied_at, applied_by)`；已应用迁移**不允许事后改内容**，md5 漂移即拒绝	版本化迁移能成立的前提
只读预览	`./migrate.sh --plan` 先看后改；裸机用 `MIGRATE_MODE=local`	存量库首次执行会把 0001 登记为已应用（内容幂等，可安全重复）
自动备份	每次 `deploy.sh` 自动 `mysqldump` 到 `deploy/backups/db-<时间戳>.sql`	备份本身落在仓库目录内是泄露面，应移出并加密
回滚锚点	`.current_tag` / `.last_good_tag` + 保留最近 3 个镜像 tag；**禁止** `docker compose down -v`	`-v` 会删 MySQL 数据卷

> **已修的历史缺陷**：`.current_tag` 与 `.last_good_tag` 曾同时写入同一个 tag，导致"回滚到刚刚构建失败的自己"——逻辑空转且让人误以为回滚已生效。现在 `rollback.sh` 对「目标 == 当前」会拒绝空回滚。

> **易混淆命名**：镜像 tag `2026.10.13-1` 与 git tag `v2026.10.13-1`（带 `v`）语义不同，回滚时容易填错。

8.6 GitHub 公开发布

8.6.1 策略：新建干净仓库，不在原仓库改历史

决策点	结论	理由
仓库策略	**新建干净仓库做首次提交**	原地改名甩不掉历史大对象；旧历史仍含旧品牌与口令注释。新仓库历史干净、体积可控，旧库完整保留可回滚
许可证	**MIT + NOTICE**，第三方源码移出仓库	ffmpeg 部分组件是 LGPL/GPL，与 MIT 不兼容；剥离 vendored 源码后用 NOTICE 说明获取方式与许可证表
素材处置	表情包、头像、课件 PDF、素材压缩包**一律不进公开仓库**	版权归属不明；真正被 UI 引用的只有 3 个，已用自绘占位图替换
脱敏位置	**只在发布仓库脱敏，源码仓库完全不动**	源码仓库要保持可运行的内部工程；发布仓库内另做清理
品牌界线	**名字可改，配置值不改**	目录名/类名/窗口标题可改；库名、容器名、证书 CN、systemd 单元名、监控 job、默认 IP 一律不改——一改线上就连不上

> **这条界线是发布方案里最容易被违反的一条**：曾有一轮把"改名""配置值变更""发布瘦身""构建改造"混在一起做，最终 8 组改动被整体回退。

8.6.2 脱敏台账（源码库残留 → 发布库处置）

类别	源码库残留	发布库
硬编码口令	4 处（三份 `TCPKernel.cpp` 注释 + 一份问题分析文档）	已清（注释重写为中性描述）
真实数据库口令	3 处（部署检查报告、口令核实手册、发布方案文档）	未入库
测试手机号	3 处	未入库
内网 IP `192.168.x`	82 处 / 32 文件（客户端源码 2 处 + 文档 80 处）	客户端 2 处改 127.0.0.1，文档整体不入库
旧品牌 `WeChat`	1593 处 / 98 文件（含 2 个 PDF 各 159 处）	源码内仅剩 2 处注释，已清

据此确立**不入公开仓库清单**：`项目类图/`、`项目总结/`（PDF/DOCX）、`问题修复文档/`、`VCC视频会议项目档案*/`、`相关文件/`、内部部署检查报告、`MSYS2/`。代价是这些目录里恰好装着最好的展示素材（80+ 张架构图、12 篇技术总结），想用要"抽图不搬库"。

8.6.3 发布仓库现状

项	值
路径 / 分支	`D:\vcc-video-conference` / `main`
提交	4 次：仓库骨架 → 补齐 LICENSE/NOTICE/README → 导入客户端与服务端源码 → 导入 deploy 部署资产
跟踪规模	231 个路径（`vcc-client/`、`server/`、`deploy/`、`docs/`、`tools/` + 根级治理文件）
治理文件	`README.md` 331 行 / `NOTICE` 107 行 / `LICENSE` MIT / `.gitignore` 125 行 / `.gitattributes`
提交身份	`Lucifar-hub095 <Lucifar-hub095@users.noreply.github.com>`（noreply，不泄露手机号）
敏感信息	真实口令 / 手机号 / 内网 IP / 密钥 **0 命中**
远端	**无 remote，未 push**

8.6.4 距"可 push"还差什么

1. **落盘工作区里未提交的审查修复**——仓库内容必须等于审查过的内容；
2. 修两个远程可利用的 P0（见 8.8）；
3. 补 `videoapi.pri` 的 `OPENCV_DIR` 空值回退，或修正 README 措辞（原文承诺了代码没做的事）；
4. README 第三方依赖清单补全：`.gitignore` 的 `*.a/*.lib` 规则曾误伤 vendored 静态库 `libopus.a` 与 `signalProcess.lib`（不在仓库里则克隆后**链接必失败**，已实测复现），现已用 `!` 例外规则放行，但 README 未提及这两个文件与 OpenCV 版本要求；
5. 定仓库名与可见性，再执行 `git remote add` + `push`（唯一不可逆的一步）。

8.6.5 push 前的四项核验

```
# ① 敏感信息三项全 0
git grep -nIE "口令|手机号" -- .   ; echo "命中数: $?"
git grep -nIE "192\\.168\\." -- .          ; echo "命中数: $?"
git ls-files | grep -E "\\.(key|pem|p12|pfx|csr|crt)$" ; echo "命中数: $?"

# ② 提交身份必须是 noreply
git log -4 --format='%an <%ae>'

# ③ 体积
du -sh .git
```

8.7 作品链接与对外呈现

8.7.1 先定性：桌面 C/S 没有网页型 demo

客户端是 Qt 桌面程序，服务端是自定义二进制协议（非 HTTP），因此**不存在"点开就能用"的网页链接**；也不建议硬造一个在线 demo——那需要长期在线服务端 + 客户端下载 + mTLS 证书分发，成本与风险远超收益。作品链接的本质不是"点进去能用"，而是**"点进去就信"**：让人在三分钟内确认「这确实是从零写的、确实能跑、架构确实清楚」。

形态	载体	作用	现状
① 仓库首页	GitHub Repo + README	第一落点，决定大部分初印象	有内容，**无截图**
② Release 分发包	GitHub Releases	证明"真的能跑"	无 tag、无 Release
③ Pages 展示页	GitHub Pages	一页式图文	未做
④ 演示视频 / GIF	README 内嵌 + 视频站链接	替代在线 demo	无素材

推荐组合：① + ② 必做；④ 做一段 60–90 秒录屏，性价比最高；③ 可选，做成"单页 + 架构图 + 截图 + 视频嵌入"，不要重写一遍 README。

8.7.2 仓库首页改造要点

现有 README 已有功能表、ASCII 架构图、13 类协议包表、构建步骤、配置方式与安全说明；缺口是**零截图、无快速体验路径、技术亮点被埋在正文、第三方依赖清单不全。**建议结构：一句话定位 → badge → 主截图/GIF → **亮点四条**（自定义协议、三层音频处理、视频链路、工程化门禁）→ 60 秒快速开始 → 架构 → 协议 → 部署 → 依赖获取 → 已知限制 → 许可与免责。

要补的素材：登录页、房间内音视频（2 张）、屏幕共享各 1 张截图，一段 ≤5MB 的双端互通 GIF，以及系统全景图与部署图各 1 张。

8.7.3 Release 与 Pages

- **Release**：tag 用语义化版本 `v0.1.0`（避免与镜像 tag `2026.10.16-3` 混淆）；附件放客户端 zip + `SHA256SUMS.txt`；**不能**附 `.env`、`certs/`、`backups/`、真实服务器 IP。二进制走 Release 附件而非入库——入库会永久污染 git 历史。
- **Pages**：需要仓库 Public（私有仓库 Pages 需付费）；建议 `/docs` 目录 + 内置主题起步，图片放 `docs/assets/`；开启后形如 `https://<账号>.github.io/<repo>/`。

8.7.4 素材复用规则：抽图不搬库

`项目类图/` 共有 **71 张 JPG**（图表汇总总览 1、系统全景 7、业务流程 3、用例 2、时序 4、ER 1、状态 5、组件 1、部署 2、活动 3、数据流 5，另有类图 35 张），配套 14 份 PDF 与 13 张 HTML 渲染页。规则：

1. **只挑 6–8 张**（全景、部署、ER、时序、数据流各 1–2 张），不要整批搬；
2. 文件名**去掉旧代号**，改成 `system-overview.jpg` / `deployment.jpg` 这类中性名；
3. 放入发布仓库 `docs/assets/diagrams/` 后引用；
4. `*.html`（mermaid 渲染页）与 `*.pdf` 不入库。

8.7.5 可直接复用的表述

- **一句话（30 字）**：基于 Qt/C++ 从零实现的桌面视频会议系统，含自定义二进制协议与音视频链路。
- **一句话（60 字）**：从零实现的桌面视频会议系统：自研二进制协议与三路通道、Opus/H.264 音视频链路、AVSync 同步模块；服务端 Docker 一键部署，本地门禁含 SQL 注入回归。
- **技术关键词**：`C++11` `Qt 5.12` `epoll` `线程池` `MySQL` `bcrypt` `FFmpeg` `H.264` `SDL2` `Opus` `WebRTC VAD` `Speex AEC` `OpenCV` `自定义二进制协议` `Docker Compose` `nginx stream` `mTLS` `CRL` `GitHub Actions`

8.8 部署与发布侧的缺陷台账

8.8.1 P0（远程可利用 / 发布阻断）

#	位置	问题	后果	状态
1	`clogic.cpp` 的 `MyMap::find` + `block_epoll_net.h`	`std::map::operator[]` 对不存在的 key **插入**默认值	未认证客户端两个普通协议包即可让 `joiner->m_userName` 空指针解引用，**远程打崩服务进程**	已修（改用迭代器查找，不存在返回 `V()`）
2	`AudioApi/audiowrite.cpp` SDL 回调	`empty()/front()` 在互斥锁外访问 `std::deque`	与主线程 push 每 20ms 并发一次，队列扩容期间锁外读取会撕裂，长期运行大概率崩溃	已修（empty/front/pop 移入同一 `lock_guard`）
3	`.gitignore` 的 `*.a` / `*.lib`	误伤 vendored `libopus.a` 与 `signalProcess.lib`	克隆后客户端链接必失败	已修（`!` 例外放行）

8.8.2 P1 摘要（按模块）

- **服务端**：构造函数不检查 `Open()` 返回值；除 H264/PLI 外不校验 `nlen` 直接强转结构体（短包越界读）；不完整包仍被分发；断开无业务层回调导致 `m_mapIDToUserInfo` 永不清理（死 fd 复用后**视频帧串线**）；重复登录只 `erase` 不 `close`；客户端 fd 无超时（slowloris 可占死线程池）；音视频帧 uid 取自包体不校验归属（**可伪造他人身份注入房间**）；`pthread_kill(tid,0)==-1` 判定错误导致死线程槽位永不复用；`epoll_wait` 持续失败时 100% CPU 忙等。
- **客户端**：包长非法仅 `continue` 不消费残留字节（连接永久解析错乱）；包头单次 `recv` 未循环收满；`nlen - 6*sizeof(int)` 前未校验 `nlen>=24`；`userGroups.erase()` 后仍访问节点引用（UB）。
- **部署脚本**：`migrate.sh` 所有 mysql 命令未带 `-D "$DB_NAME"`（**迁移完全无法运行**）；`verify.py` 第 4 步 `finally` 里 `bad.close()` 抛 NameError（正常场景误报失败）；`pack_client.bat` 的 `if "%~1:~0,2%"` 不是合法子串展开（`--xxx` 被当成 IP）；`rollback.sh` 回滚验证写生产库。

8.8.3 部署运维待办

优先级	事项	验收
P1	修 `migrate.sh` 缺 `-D "$DB_NAME"`	`./migrate.sh --plan` 不再报 "No database selected"
P1	修 `verify.py` 的 `bad.close()` NameError	`--expect-fail` 场景返回 0
P1	`rollback.sh` 回滚验证改走隔离栈	回滚过程不再新增 `t_user` 行
P2	`.env` 权限收紧	必须用 `icacls /inheritance:r` —— `chmod 600` 在 NTFS 上**静默失效**（实测仍是 0666）
P2	依赖锁 digest（`mysql:8.0` / `nginx:1.27-alpine`）	两次构建镜像 digest 一致
P2	接入监控告警	`alert-rules.yml` 目前只有规则、无 exporter，实际零监控

8.8.4 已确认完成的项

容器化部署跑通（三容器 healthy）、明文端口收敛到回环、隔离冒烟零污染、业务级健康探针、`schema.sql` 幂等、版本化迁移 + md5 一致性、TLS 反向用例误报修复、登录包 memset 越界修复、`UpdateEmailRq` 越权修复、发布库敏感扫描 0 命中、提交邮箱改 noreply、版权素材移出并替换占位图、客户端全量构建成功。